INTRODUCTION TO MICROCOMPUTER
2025-2026 FALL
TERM PROJECT

Definitions and Rules:

1. Within the group work,
   - **i.** All group members must be involved in the design and development process.
   - **ii.** Tasks must be distributed to all group members, and this distribution must be clearly stated in the report.
   - **iii.** Each student will have an account on GitHub, which serves as a code version tracking system. A team leader will be selected for each group. The leader will create a project. Team members will work on the same project using the source files assigned to them. Each team member's performance in this project will be considered in the evaluation.
   - **iv.** Each member must write the assigned code and test programs. Each source file must include the name of the person who wrote the code.
2. The commands explaining the code segments must be written on the source codes.
3. A report must be prepared for the project (A template has been given in A.1 below), outlining the design and implementation phases.
4. The project report must be uploaded to ESUZEM before the deadline. You must compress the written code and project report as a .zip or .rar file, name the file "Name_of_One_Group_Member.zip/rar", and upload it to the ESUZEM.
5. **<u>All the codes which will be run on PIC16F877A must be written in assembly language</u>**.
6. **The codes which will be run on PC may be written any high-level languages (C/C++, python, etc.).**
7. **<u>If your design and/or codes are partially or completely taken from other groups or unreferenced sources, you will receive a grade of zero.</u>**
8. **<u>Project groups are required to make presentations at the end of the semester.</u>**

TITLE: HOME AUTOMATION

Abstract

The project aims to develop a system that will control sensors and drivers for home automation (such as temperature control, curtain opening and closing, and ambient temperature and humidity display) using PIC microcontrollers. It will also allow users to manage and monitor the system from their personal computers.

To achieve this goal, the system will consist of two parts. The first part requires developing programs to manage the peripherals (sensors, drivers, displays, etc.) connected to the inputs and outputs of two PIC16F877A microcontrollers. The second part involves developing programs to be run on the PC connected to the PIC16F877A microcontroller via the serial (UART) communication interface. On the PC side, an application development library (API) and a sample application are expected to be written to manage the peripherals connected to the microcontrollers.

1. Introduction

A simulator will be used to implement the system [1], [2]. PICSimLab is a free and open-source simulator that can be installed on a computer running Linux, Windows, or MacOS [1], [2]. The system to be simulated is shown in Figure 1. The gpboard board will be used in the simulator. The peripherals differ for Board #1 and Board #2. By adding the peripheral components shown in Figure 1, the connections of each component to the PIC pins must be defined. Once the system shown in Figure 1 is created, it can be saved. Then, each member of the team can work on writing and testing code blocks.

A serial communication emulator can be used to establish a UART connection between the PICSimLab software and the program that will be run on the PC [3]-[5]. The emulator creates ports at both ends for serial communication, for example, let's say COM1 and COM2. One of these ports (say COM1) is used by the program running on the PC, while the other (say COM2) is used by the PICSimLab (via the IO UART Module). Thus, data sent from both sides through these ports is transmitted to each other by the emulator.

The software architecture is given in Figure 2. The detailed explanations about the code blocks seen in the figure are given in Chapter 2.

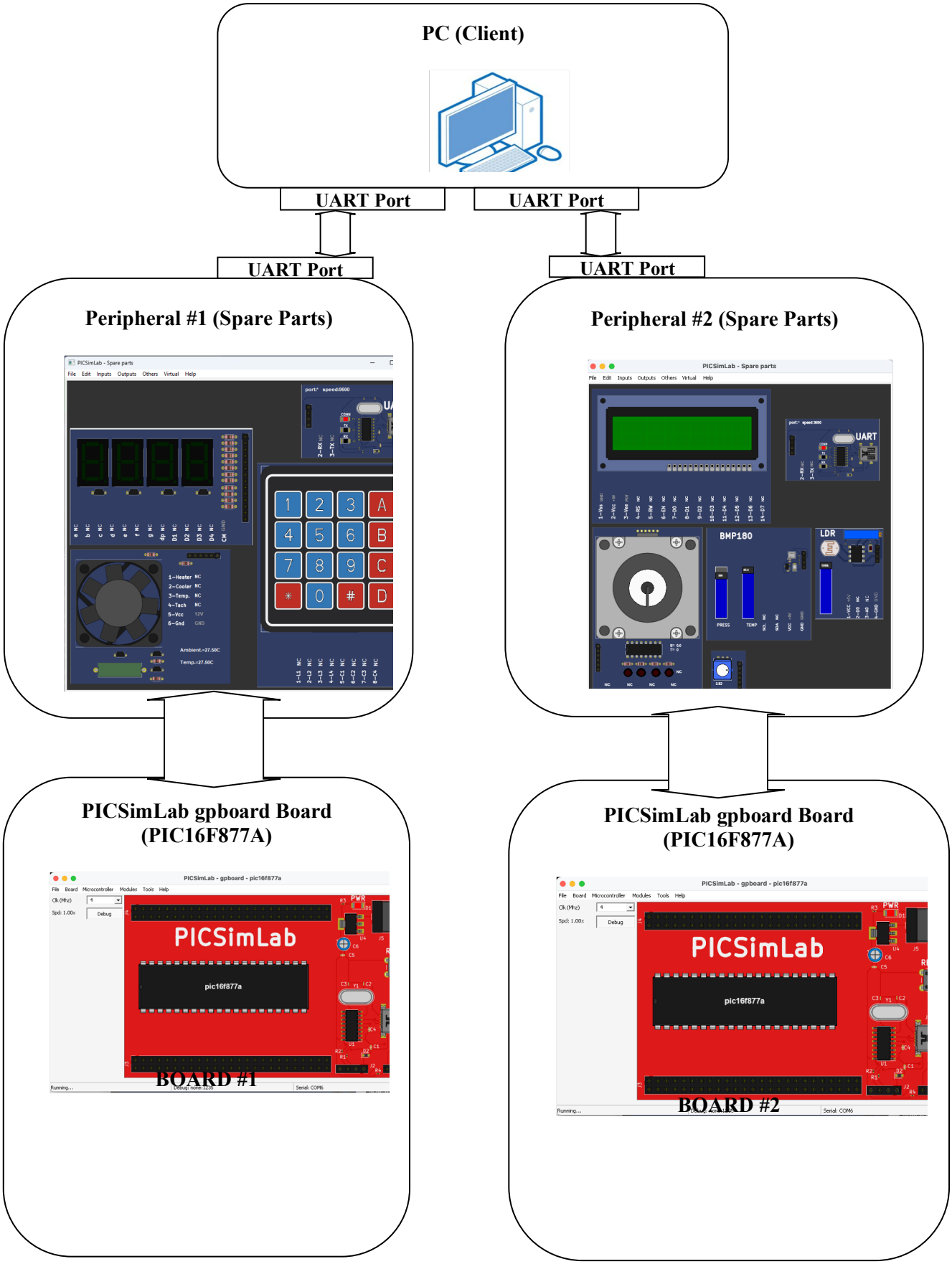


Figure 1. Hardware Architecture

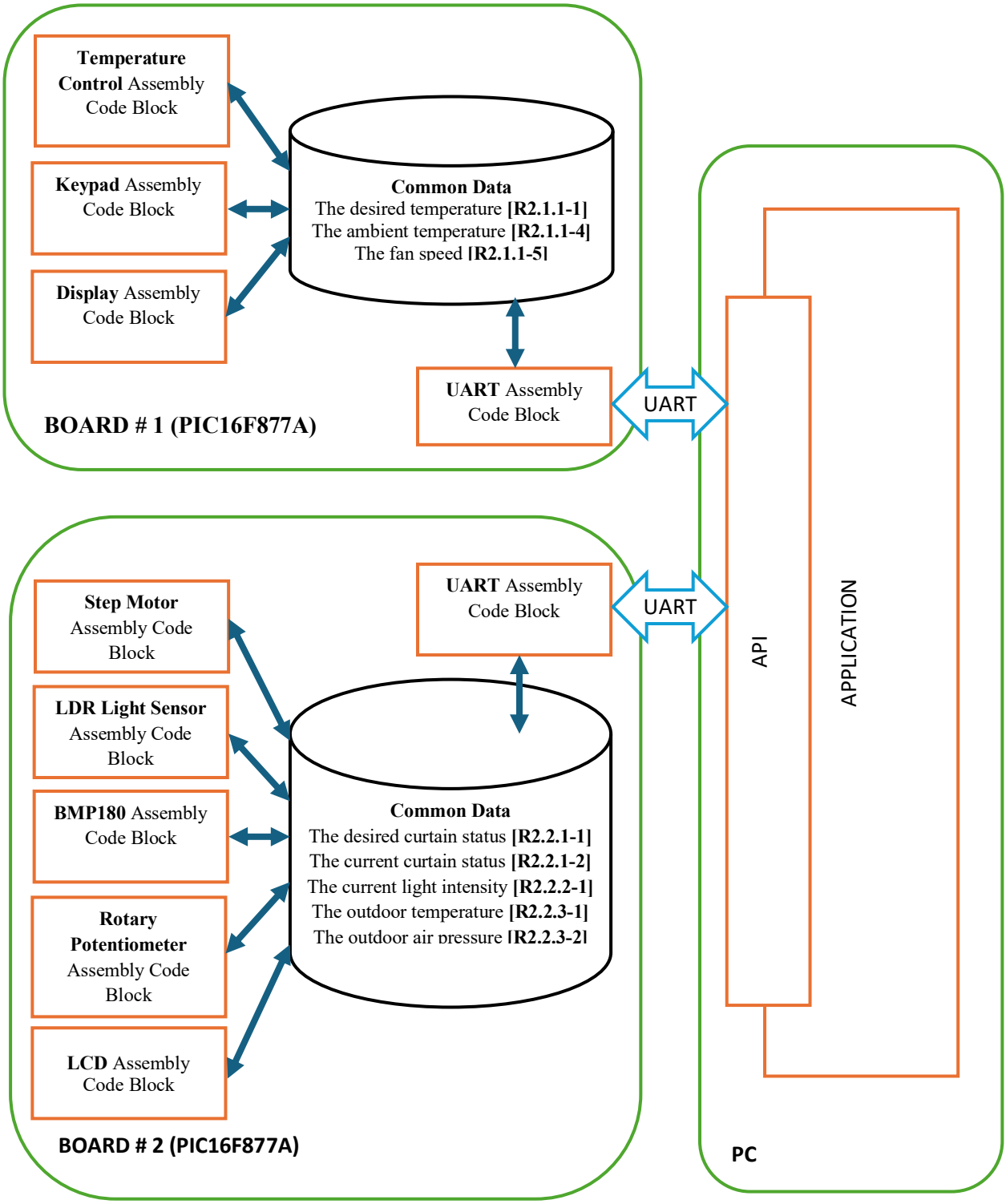


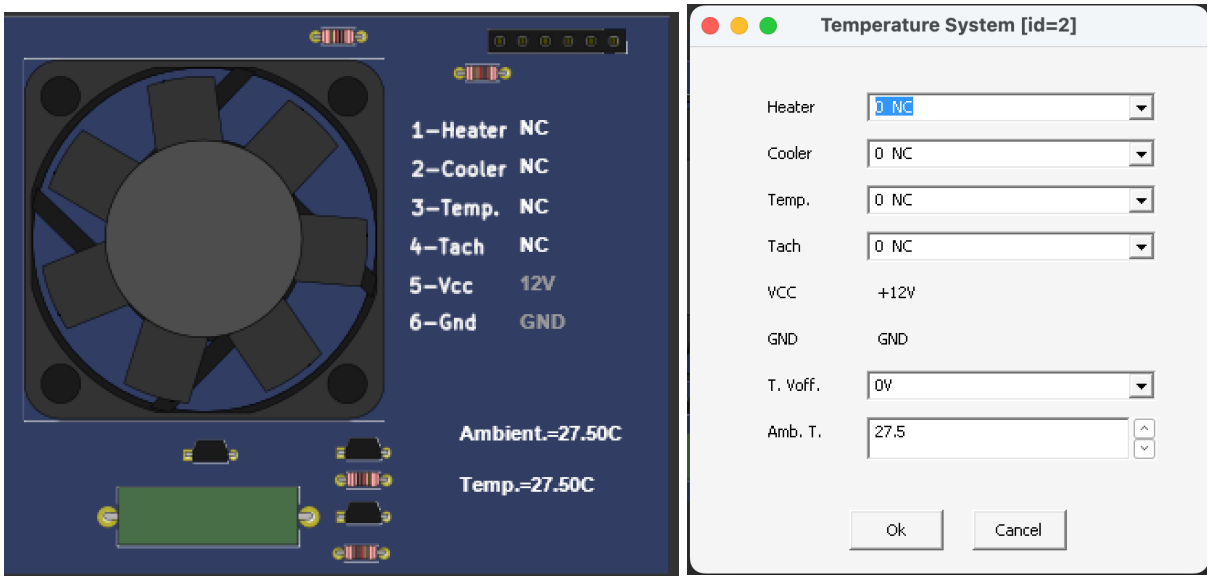
Figure 2. Software Architecture

1.1 Peripherals

There are different peripherals managed by each board. You can find detailed explanations and sample programs for these in the PICSimLab online documentation.

1.1.1 Temperature System

This module is a temperature control system. The temperature control system consists of a heating resistor, an LM35 temperature sensor, a cooler, and an infrared tachometer.



(a) (b)

Figure 3. Temperature Control System Module of PICSimLab: (a) GUI, (b) Property Setting Window

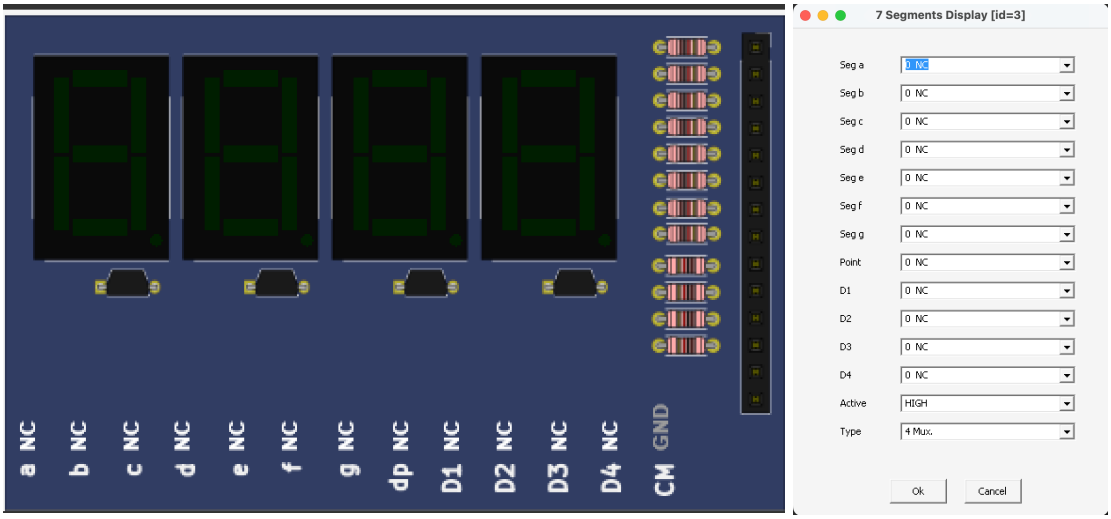
Component	Interface Type (PIC16F877A Pin Type)	Definition
Heater	Digital Output	Open/Close heater
Cooler	Digital Output	Open/Close cooler
Temp	Analog Input	Read the current temperature
Tach	Timer external clock input	Tachometer produces a pulse at each turn of cooler fan. If you count the pulses for a second, you get the speed of the fan in rps (rotation per second)

For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/TemperatureSystem.html

1.1.2 7-Segment Display

This module can be configured as four multiplexed or one single 7-segment display.



(a) (b)

Figure 4. 7-Segment Display Module of PICSimLab: (a) GUI, (b) Property Setting Window

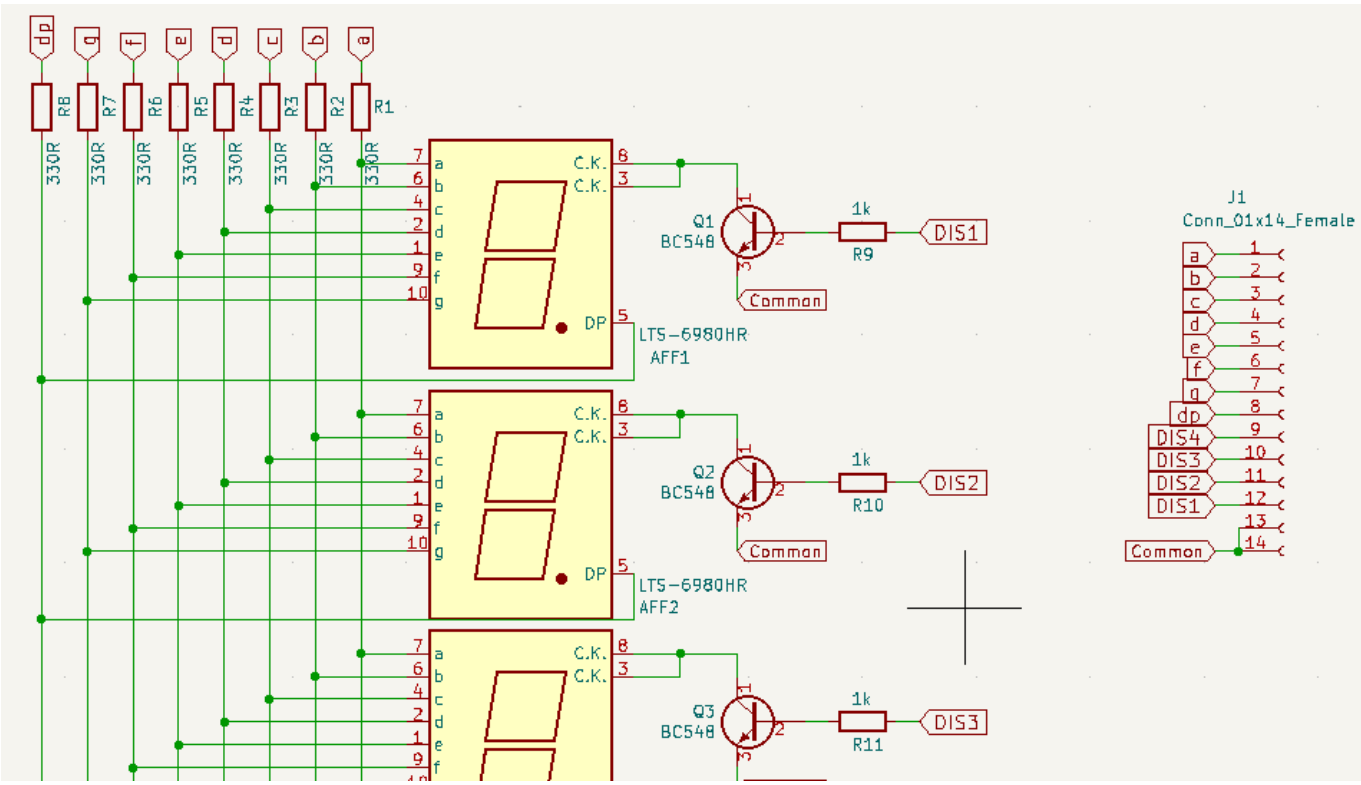


Figure 5. Circuit Diagram of 7-Segment Display Module

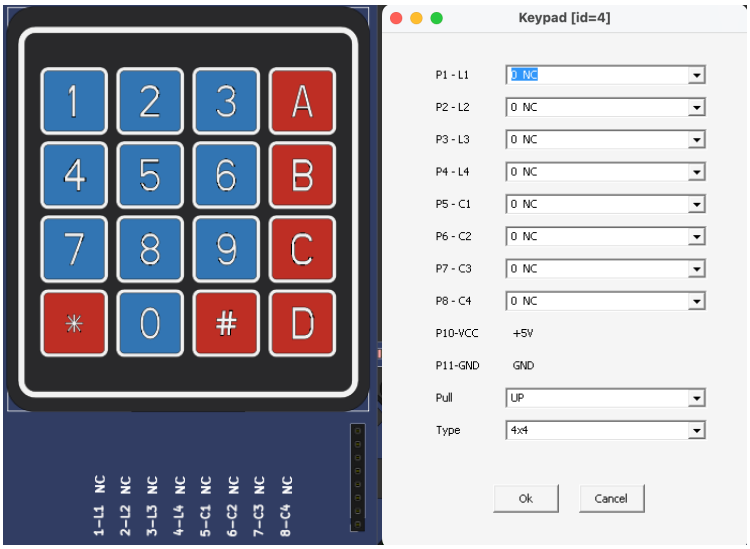
Component	Interface Type (PIC16F877A Pin Type)	Definition
Seg a-g	Digital Outputs	Display any number
Point (dp)	Digital Output	Display point
D1-D4	Digital Outputs	Select one of the displays

For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/7SegmentsDisplay.html

1.1.3 Keypad

This module features a matrix keyboard that is configurable to 4x3, 4x4, or 2x5 rows/columns.



(a) (b)

Figure 6. Keypad Module of PICSimLab: (a) GUI, (b) Property Setting Window

Component	Interface Type (PIC16F877A Pin Type)	Definition
P1-L1 – P4-L4	Digital Inputs	Determination of the key pressed.
P5-C1 – P8-C4	Digital Outputs	Determination of the key pressed.

For more help, you can visit the link:

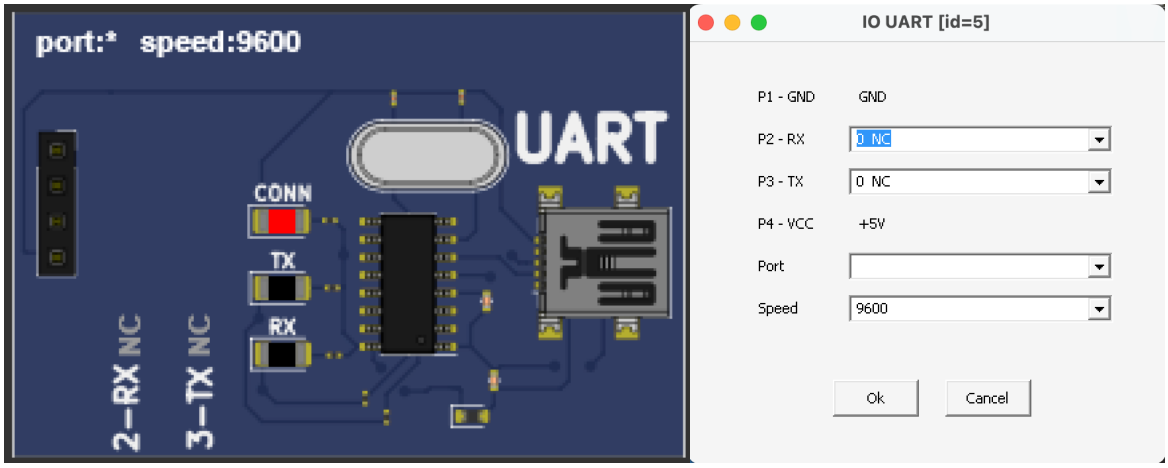
https://lcgamboa.github.io/picsimlab_docs/0.9.2/Keypad.html#x86-890009.2.14

1.1.4 UART IO

This module is a UART serial port. This part connects the hardware/software UART IO pins of the microcontroller to one real/virtual PC serial port. To use a virtual port, you need to install a virtual port software, as described in

(https://lcgamboa.github.io/picsimlab_docs/0.9.2/SerialCommunication.html#def_seriali).

The serial communication uses the 8N1 format.



(a) (b)

Figure 7. UART IO Module of PICSimLab: (a) GUI, (b) Property Setting Window

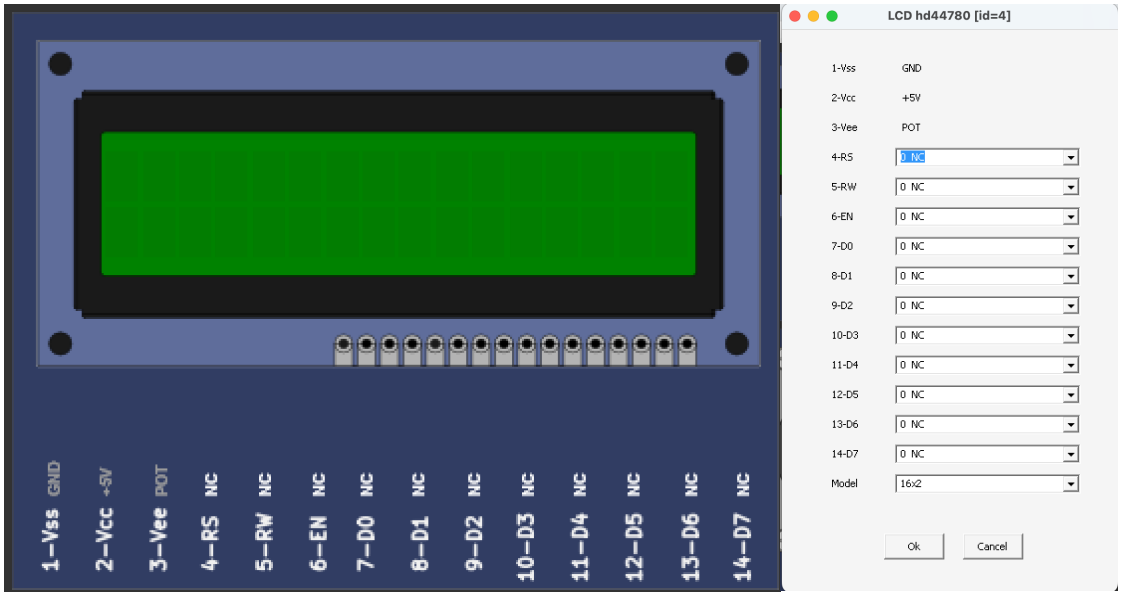
Component	Interface Type (PIC16F877A Pin Type)	Definition
P2-RX	Transmit (TX)	Send data via UART
P3-TX	Receive (RX)	Receive data via UART

For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/IOUART.html

1.1.5 LCD hd44780

This module is a text display with 2 (or 4) lines by 16 (or 20) columns.



(a) (b)

Figure 8. LCD hd44780 Module of PICSimLab: (a) GUI, (b) Property Setting Window

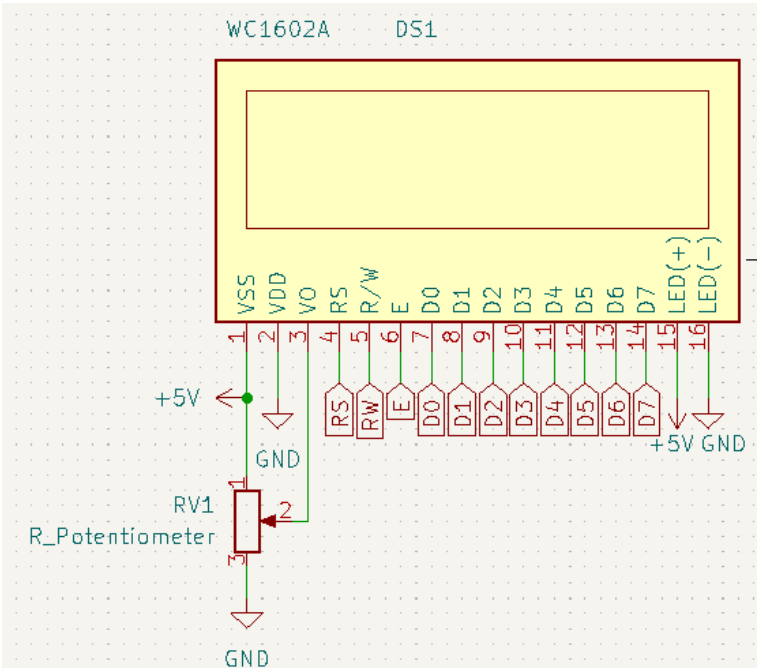


Figure 9. Circuit Diagram of LCD Module

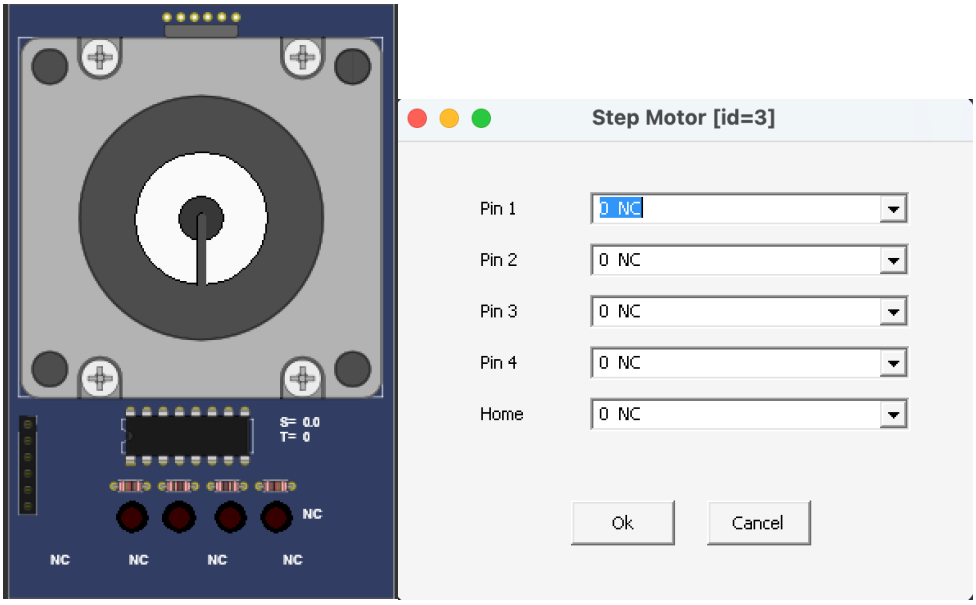
Component	Interface Type (PIC16F877A Pin Type)	Definition
D0-D7	Digital Outputs	Databus
EN	Digital Output	Enable
RS	Digital Output	Register select
R/W	GND	Ground

For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/LCDhd44780.html

1.1.6 Step Motor

This module is a component with 4 coils that must be driven in the correct order to rotate the rotor. Each step of the motor is 1.8º.



(a) (b)

Figure 10. Step Motor Module of PICSimLab: (a) GUI, (b) Property Setting Window

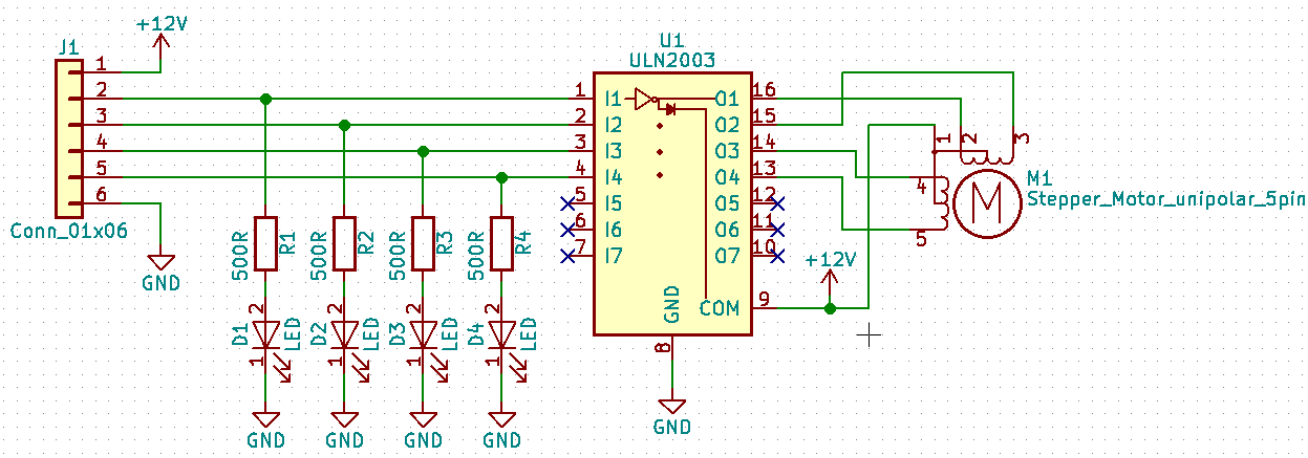


Figure 11. Circuit Diagram of Step Motor Module

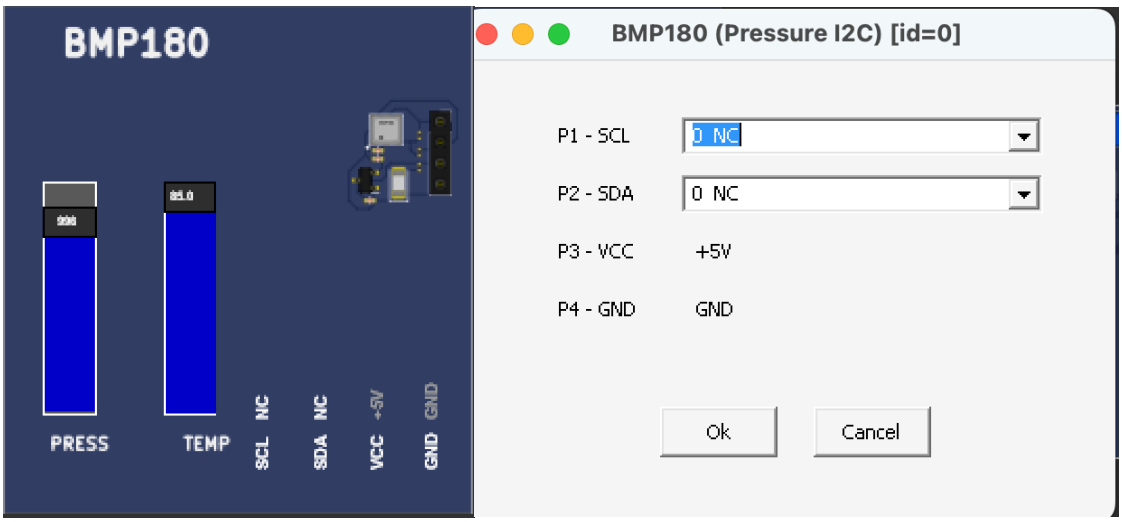
Component	Interface Type (PIC16F877A Pin Type)	Definition
Pin 1-4	Digital Outputs	Rotates the motor

For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/StepMotor.html

1.1.7 BMP180

This module is BMP180 I2C pressure and temperature sensor (address 0x77). The temperature range is -40 to 85 °C, and the pressure range is 300 hPa to 1100 hPa.



(a) (b)

Figure 12. BMP180 Module of PICSimLab: (a) GUI, (b) Property Setting Window

Component	Interface Type (PIC16F877A Pin Type)	Definition
SCL	SCL (I^2C pin)	Read pressure and temperature via Serial communication
SDA	SDA (I^2C pin)	Read pressure and temperature via Serial communication

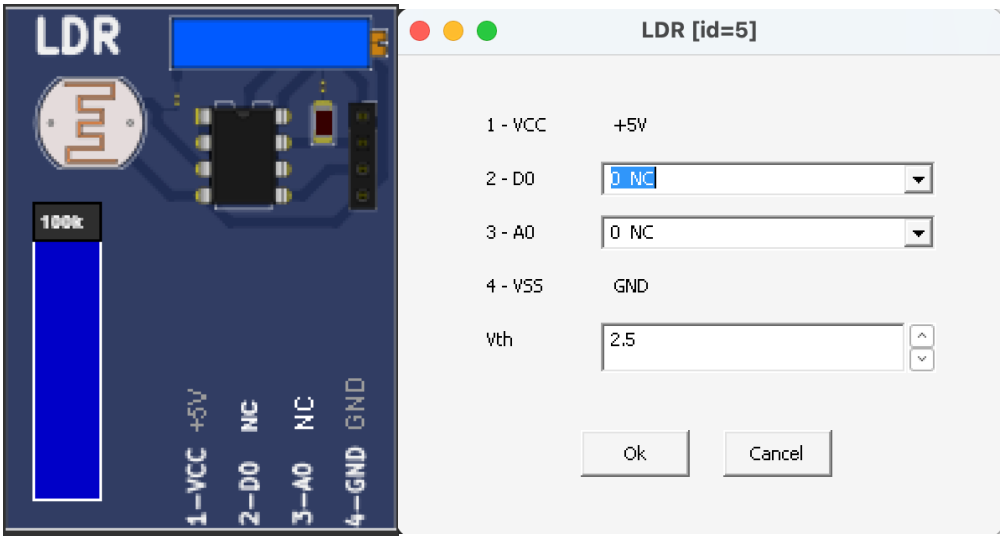
For more help, you can visit the link:

https://lcgamboa.github.io/picsimlab_docs/0.9.2/BMP180PressureI2C.html

1.1.8 LDR Light Sensor

This module is light light-dependent resistor (LDR) connected in series with a 10K resistor. The analog output of the voltage divider is applied to one voltage follower and can be read directly from pin A0. The analog value from the voltage follower is compared with one voltage threshold, the digital output of the comparator, and can be read directly from pin D0.

LDR Characteristics	Value
Gamma value at 100-10Lux	0.7
Light Resistance at 10Lux (25°C)	20KΩ



(a) (b)

Figure 13. LDR Light Sensor Module of PICSimLab: (a) GUI, (b) Property Setting Window

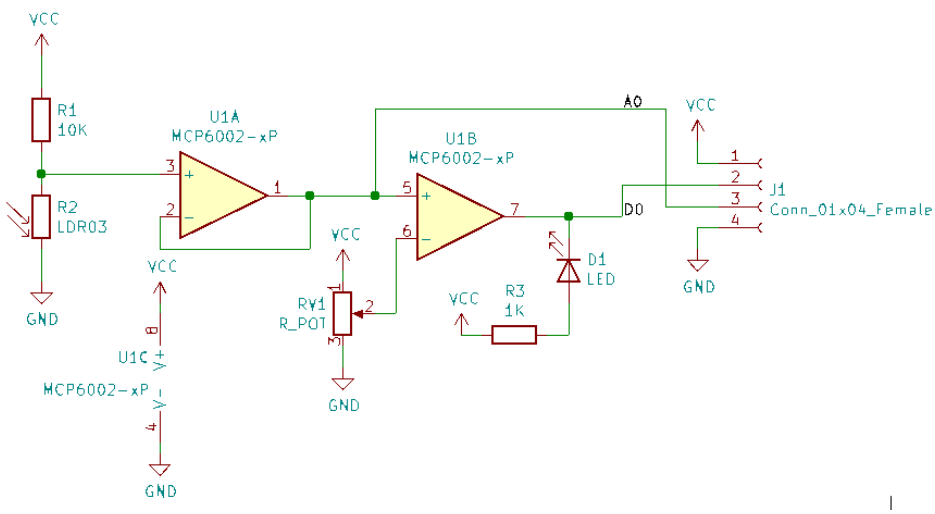


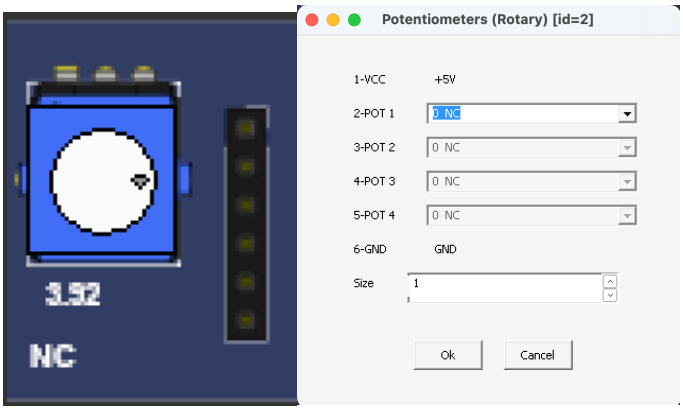
Figure 14. Circuit Diagram of LDR Module

Component	Interface Type (PIC16F877A Pin Type)	Definition
D0	Digital Input	Read the comparison of light intensity with a threshold
A0	Analog Input	Read the voltage related to the light intensity

For more help, you can visit the link:
https://lcgamboa.github.io/picsimlab_docs/0.9.2/LDR.html

1.1.9 Potentiometers (Rotary)

This module is light light-dependent resistor (LDR) connected in series with a 10K resistor. The analog output of the voltage divider is applied to one voltage follower and can be read directly from pin A0. The analog value from the voltage follower is compared with one voltage threshold, the digital output of the comparator, and can be read directly from pin D0.



(a) (b)

Figure 15. Potentiometers (Rotary) Module of PICSimLab: (a) GUI, (b) Property Setting Window

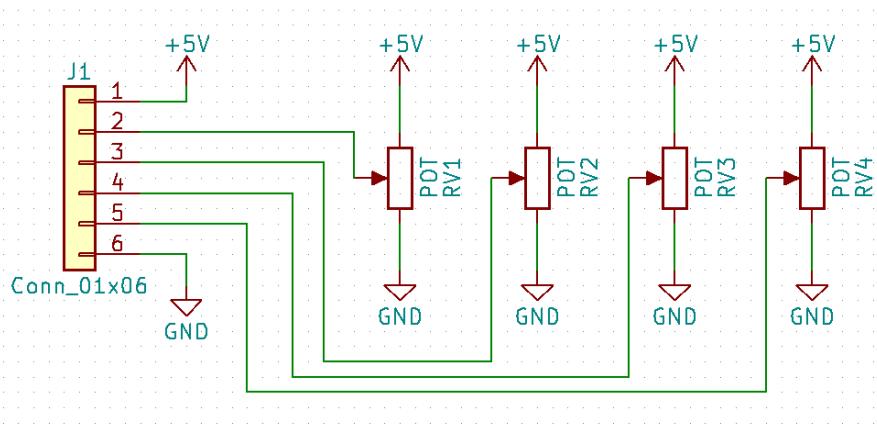


Figure 16. Circuit Diagram of Potentiometers (Rotary) Module

Component	Interface Type (PIC16F877A Pin Type)	Definition
POT1	Analog Input	Read the analog value of potentiometer

For more help, you can visit the link:
https://lcgamboa.github.io/picsimlab_docs/0.9.2/PotentiometersRotary.html

2. Requirements

The requirements for the software that you will develop will be defined in a modular format. The same modular structure can be used throughout the development process, and team members can share modules.

Common requirements for the entire system are:

- **[R2_1]** Which pins of the peripherals are connected to which pins of the microcontroller should be explicitly given in a table.

2.1 Requirements for Board #1 and Peripheral #1 (Home Air Conditioner System)

The board #1 with its peripherals #1 manages the air conditioning system of the house.

2.1.1 Temperature Control Module

The temperature control will be done.

- **[R2.1.1-1]** A memory address is reserved to keep **the desired temperature** value (both integral and fractional parts of the floating number).
- **[R2.1.1-2]** If the user-defined desired temperature value is greater than the ambient temperature, the heater will turn off, and the fan will turn on.
- **[R2.1.1-3]** If the user-defined desired temperature value is less than the ambient temperature, the heater will turn on, and the fan will turn off.
- **[R2.1.1-4] The ambient temperature** value will be read periodically and kept updated at a memory address.
- **[R2.1.1-5]** The fan speed (in rps) value will be read periodically and kept updated at a memory address.

2.1.2 Keypad Module

The user's desired temperature value will be entered.

- **[R2.1.2-1]** Before entering the desired temperature value, the user must press button A. Pressing button A will run the interrupt routine.
- **[R2.1.2-2]** The user will enter the desired temperature value, with a maximum of two digits of the integral part and one digit of the fractional part, by pressing the buttons. After entering the number, the user will press the button '#'. For the decimal point, the user will press the button '*'. For example, to set the desired temperature value 29.5, one presses the buttons 'A', '2', '9', '*', '5', and '#', respectively.
- **[R2.1.2-3]** If the user enters a desired temperature value that is less than 10.0 and greater than 50.0, then the value is rejected and not saved into the memory.
- **[R2.1.2-4]** The desired temperature value in the accepted ranges, which is entered by keypad, will be kept updated at a memory address. The memory address will be the same as **[R2.1.1-1].**

2.1.3 Display Module

The desired temperature, ambient temperature, and fan speed are displayed.

- **[R2.1.3-1]** The desired temperature (at the memory address same as **[R2.1.1-1]**), ambient temperature (at the memory address same as **[R2.1.1-4]**), and fan speed (at the memory address same as **[R2.1.1-5]**) will be shown on the display at 2-second intervals.

2.1.4 UART Module

Some level of management of the Home Air Conditioner System is served via the UART module.

- **[R2.1.4-1]** Requests arriving via the UART port will be serviced. These requests will be handled using the commands below or a command set you specify.

Definition	Message received by Module	Message sent by Module	Action done by Module
Get desired temperature low byte (fractional part)	00000001B	<desired temperature low byte (fractional part)>	Reads fractional part of desired temperature (at the memory address same as **[R2.1.1-1]**) and sends over port.
Get desired temperature high byte (integral part)	00000010B	< desired temperature high byte (integral part)>	Reads integral part of desired temperature (at the memory address same as **[R2.1.1-1]**) and sends over port.
Get ambient temperature low byte (fractional part)	00000011B	<desired temperature low byte (fractional part)>	Reads fractional part of ambient temperature (at the memory address same as **[R2.1.1-4]**) and sends over port.
Get ambient temperature high byte (integral part)	00000100B	< desired temperature high byte (integral part)>	Reads integral part of ambient temperature (at the memory address same as **[R2.1.1-4]**) and sends over port.
Get fan speed (rps)	00000101B	< fan speed byte (rps)>	Reads fan speed (at the memory address same as **[R2.1.1-5]** and sends over port.
set desired temperature low byte (fractional part)	$10t_5t_4t_3t_2t_1t_0$B	None	$t_5t_4t_3t_2t_1t_0$ is the desired temperature in 6-bit binary form. Parse the 6-bit number (fractional part) and save to the memory address (at the memory address same as **[R2.1.1-1]**)
set desired temperature high byte (integral part)	$11t_5t_4t_3t_2t_1t_0$B	None	$t_5t_4t_3t_2t_1t_0$ is the desired temperature in 6-bit binary form. Parse the 6-bit number (integtal part) and save to the memory address (at the memory address same as **[R2.1.1-1]**)

2.2 Requirements for Board #2 and Peripheral #2 (Curtain Control System)

The board #2 with its peripherals #2 manages the curtain control system of the house.

2.2.1 Step Motor Module

By rotating the step motor clockwise (CW) and counterclockwise (CCW), the curtain opens and closes, respectively. Assume that the curtain becomes fully open and fully closed by the five full turns of the step motor. Since each step of the motor is 1.8$^{o\,\circ}$, one full turn of the step motor becomes 200 steps. If you assume that the curtain is fully open and the step motor is zero position, then the curtain becomes fully closed by rotating the motor 200*5=1000 steps CCW. Similarly, the curtain will be fully open by turning the motor 1000 steps CW. The curtain status will be 0% for fully open and 100% for fully closed. Thus, the relation between the step and curtain status becomes 10 step/%. Thus, for a given openness ratio of the curtain (between 0% and 100%), one can calculate how many steps the motor should rotate.

- **[R2.2.1-1]** A memory address is reserved to keep **the desired curtain status** value (in percent).
- **[R2.2.1-2]** A memory address is reserved to keep **the current curtain status** value (in percent), and the status value is updated with the rotation of the step motor.
- **[R2.2.1-3]** When the desired curtain status is different from the current curtain status, the direction of movement (CW or CCW) and step value must be calculated, and the step motor must be rotated.

2.2.2 LDR Light Sensor Module

This sensor measures the light intensity of the room and manages the curtain status by considering the light intensity.

- **[R2.2.2-1]** A memory address will be reserved to keep **the current light intensity** value, and the value will be updated when the light intensity changes.
- **[R2.2.2-2]** When the light intensity falls below the threshold value, the desired curtain status of 100% will be assigned to the memory address (at the memory address same as **[R2.2.1-1]**) to fully close the curtain.

2.2.3 BMP180 Module

This sensor measures the pressure and temperature of the outdoor.

- **[R2.2.3-1]** A memory address will be reserved to keep **the outdoor temperature** value, and the value will be updated when the temperature changes.
- **[R2.2.3-2]** A memory address will be reserved to keep **the outdoor air pressure** value, and the value will be updated when the pressure changes.

2.2.4 Rotary Potentiometer Module

This module is a rotary switch which is rotated by the user to change the status of the curtains between 0% and 100%. By rotating the switch, its output changes linearly between 0 and 5. By rotating the switch from 0 to 5, the curtain status will change linearly from 0% to 100%.

- **[R2.2.4-1]** When the switch is rotated and moved to a certain position, the desired curtain value corresponding to that position will be saved into a memory address (at the memory address same as **[R2.2.1-1]).**

2.2.5 LCD Module

The outdoor temperature, outdoor pressure, light intensity, current curtain status are displayed.

- **[R2.2.5-1]** The outdoor temperature (at the memory address same as **[R2.2.3-1]**), outdoor pressure (at the memory address same as **[R2.2.3-2]**), light intensity (at the memory address same as **[R2.2.2-1]**) and current curtain status (at the memory address same as **[R2.2.1-2]**) will be shown on the 2-line and 16-column LCD display as follows:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Line 1	sign	xt	xt	.	xt	°	C			xp	xp	xp	xp	h	P	a
Line 2	xl	xl	xl	xl	xl	L	u	x		xc	xc	.	xc	%		

* xt, digits of temperature; xp, digits of pressure; xl, digits of light; xc, digits of curtain

2.2.6 UART Module

Some level of management of the Curtain Control System is served via the UART module.

- **[R2.2.6-1]** Requests arriving via the UART port will be serviced. These requests will be handled using the commands below or a command set you specify.

Definition	Message received by Module	Message sent by Module	Action done by Module
Get desired curtain status low byte (fractional part)	00000001B	<desired curtain low byte (fractional part)>	Reads fractional part of desired curtain (at the memory address same as **[R2.2.1-1]**) and sends over port.
Get desired curtain high byte (integral part)	00000010B	< desired curtain high byte (integral part)>	Reads integral part of desired curtain (at the memory address same as **[R2.2.1-1]**) and sends over port.
Get outdoor temperature low byte (fractional part)	00000011B	<outdoor temperature low byte (fractional part)>	Reads fractional part of outdoor temperature (at the memory address same as **[R2.2.3-1]**) and sends over port.
Get outdoor temperature high byte (integral part)	00000100B	< outdoor temperature high byte (integral part)>	Reads integral part of outdoor temperature (at the memory address same as **[R2.2.3-1]**) and sends over port.
Get outdoor pressure low byte (fractional part)	00000101B	< outdoor pressure low byte (fractional part)>	Reads fractional part of outdoor pressure (at the memory address same as **[R2.2.3-2]**) and sends over port.
Get outdoor pressure high byte (integral part)	00000110B	< outdoor pressure high byte (integral part)>	Reads integral part of outdoor pressure (at the memory address same as **[R2.2.3-2]**) and sends over port.
Get light intensity low byte (fractional part)	00000111B	<light intensity low byte (fractional part)>	Reads fractional part of light intensity (at the memory address same as **[R2.2.2-1]**) and sends over port.
Get light intensity high byte (integral part)	000010000B	< light intensity high byte (integral part)>	Reads integral part of light intensity (at the memory address same as **[R2.2.2-1]**) and sends over port.
set desired curtain status low byte (fractional part)	$10c_5c_4c_3c_2c_1c_0$B	None	$c_5c_4c_3c_2c_1c_0$ is the desired curtain status in 6-bit binary form. Parse the 6-bit number (fractional part) and save to the memory address (at the memory address same as **[R2.2.1-1]**)
set desired curtain status high byte (integral part)	$11c_5c_4c_3c_2c_1c_0$B	None	$c_5c_4c_3c_2c_1c_0$ is the desired curtain status in 6-bit binary form. Parse the 6-bit number (integtal part) and save to the memory address (at the memory address same as **[R2.2.1-1]**)

2.3 Requirements for Application Programming Interface (API) (PC Side, with high level languages like C/C++, Python, Java, etc.)

The API will provide high level functions encapsulating the serial communication with the board #1 and #2.

- **[R2.3-1]** Classes that will send and receive data to/from both boards (PIC16F877A) must be improved similar to shown in the figure below. Class members can be modified as needed.
- **[R2.3-2]** To test each member functions of classes, test program should be written.

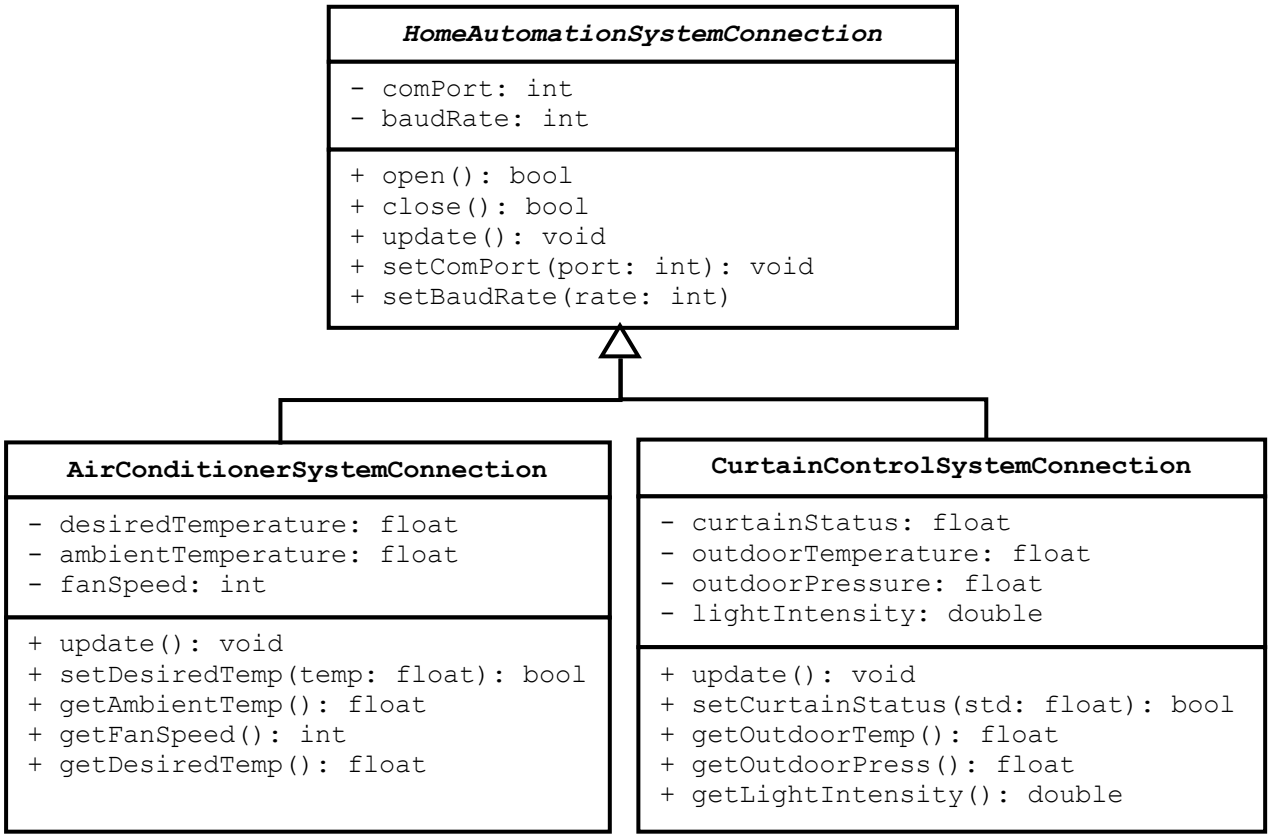


Figure 17. UML Class Diagram for API

Member Functions	**Definitions**
`open`	Initiate a connection to the Board via UART port with the defined port and baud rate.
`close`	Closes the connection to the board.
`update`	By sending and receiving messages over connection, gets all data and updates the member data (`desiredTemperature, curtainStatus, etc`)
`setComPort`	Set the communication port number
`setBaudRate`	Set the communication baudrate
`setDesiredTemp`	Set the desired temperature by sending message to the board.
`setCurtainStatus`	Set the desired curtain status by sending message to the board.
`getAmbientTemp`	Get the ambient temperature
`getFanSpeed`	Get the fan speed
`getDesiredTemp`	Get the desired temperature
`getOutdoorTemp`	Get the outdoor temperature
`getOutdoorPress`	Get the outdoor pressure
`getLightIntensity`	Get the light intensity

2.4 Requirements for Application (PC Side, with high level languages like C/C++, Python, Java, etc.)

The application is a program written by using the API (developed in **[R2.3-1]**). The application provides a menu in order to manage the home automation system. By using menus, the user can see the home data on screen and sets the home temperature and curtain openness.

- **[R2.4-1]** An application must be developed to include some menus. The user can choose any item in the menu to show some system data and set the temperature and curtain openness. The menu and functions must contain all items shown in Figure below. You can write by any programming language. The application may be GUI (graphical user interface)-based or console-based, choose yourself which one you want to.

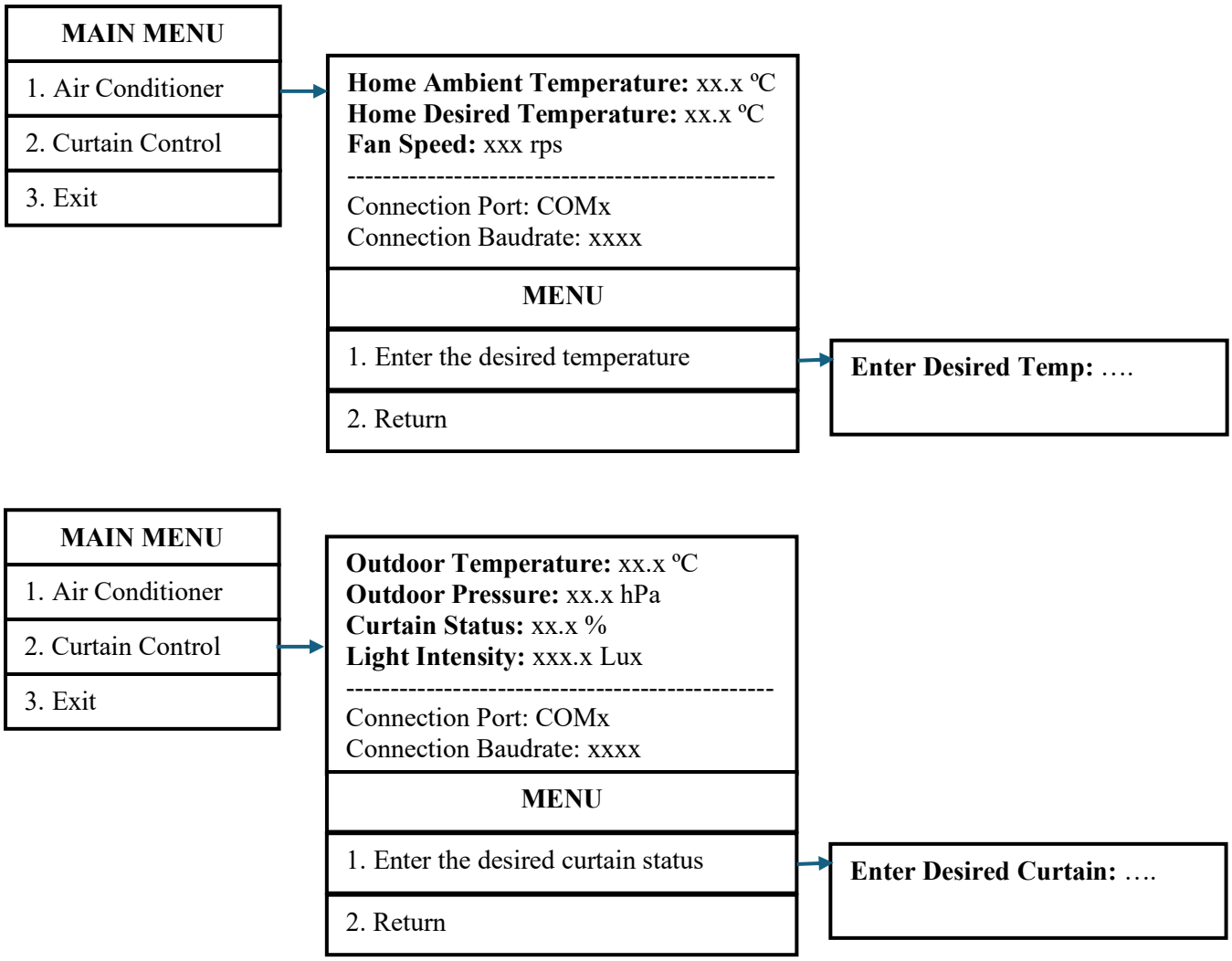


Figure 18. Application Sample Menu Operations

References

[1] L.C.G. Lopes. PICsimLab-Programmable IC Simulator Laboratory. https://github.com/lcgamboa/picsimlab.

[2] PICSimLab, https://sourceforge.net/projects/picsim/, Last access: 28 October 2025.

[3] SerialPortEmilator, https://lcgamboa.github.io/picsimlab_docs/stable/SerialCommunication.html#def_seriali, Last access: 28 October 2025.

[4] com0com, http://sourceforge.net/projects/com0com/, Last access: 28 October 2025.

[5] tty0tty, https://github.com/lcgamboa/tty0tty, Last access: 28 October 2025.

A.1 Report Format

Cover Page

T.R.
ESKISEHIR OSMANGAZI UNIVERSITY
DEPARTMENT OF ELECTRICAL-ELECTRONICS ENGINEERING
AND
DEPARTMENT OF COMPUTER ENGINEERING

INTRODUCTION TO MICROCOMPUTERS
TERM PROJECT

Project Title

Project Group Member ID and Name, department
1512xxxxxxxx, Ali VELİ, Electrical and Electronics Engineering
1521xxxxxxxx, Ahmet MEHMET, Computer Engineering …

January 2026

Report Contents

1. **Introduction**

   General information about project

2. **Design**

   You may add subtitles. This part includes explanations about design, UML diagrams, block diagrams, task assignments, sample outputs of the application program for the sample inputs, etc.

Task Assignments

Group Member	Tasks
Name of Group member	. . .
Name of Group member	. . .
. . .	. . .

3. **Conclusion**

   Evaluation of the project results, comments about the teamwork, pros and cons, advice for further works, etc.

A.2 Checklist

CHECK LIST

List	**Included/Done**
All requirements (R2_1, R2.1.1-1, R2.1.1-2, R2.1.1-3, R2.1.1-4, R2.1.1-5, R2.1.2-1, R2.1.2-2, R2.1.2-3, R2.1.2-4, R2.1.3-1, R2.1.4-1, R2.2.1-1, R2.2.1-2, R2.2.1-3, R2.2.2-1, R2.2.2-2, R2.2.3-1, R2.2.3-2, R2.2.4-1, R2.2.5-1, R2.2.6-1, R2.3-1, R2.3-2, R2.4-2	Yes / No
Codes are well formatted	Yes / No
Codes include commands	Yes / No
Source codes of API	Yes / No
There are test programs for API	Yes / No
Source Codes of Application	Yes / No
Assembly Codes for Board #1	Yes / No
Assembly Codes for Board #2	Yes / No
Report	Yes / No

GOOD LUCKS…